MSA University
Faculty of Engineering

SLIM Lightweight Block Cipher

CSE461 Computer Security Project Report

Student Name	Student ID	Responsibility
Fares Ahmed	236003	Algorithm implementation and encryption/decryption functions
Kerolos Ayman	232791	Testing, screenshots, and report preparation
Mostafa Mohamed	235719	Algorithm implementation and encryption/decryption functions

1. Introduction

Lightweight cryptography is used when security is needed on devices with limited memory, processing power, and energy. Examples include RFID tags, Internet of Things devices, wearable sensors, and healthcare monitoring systems. These systems cannot always use heavy cryptographic algorithms, so lightweight block ciphers use simple operations such as XOR, substitution, permutation, and compact key scheduling.

This project implements an educational SLIM-style lightweight block cipher in Python. The program accepts normal plaintext from the user, converts it into 16-bit blocks, encrypts each block using a Feistel structure, and then decrypts the ciphertext to recover the original plaintext.

2. Algorithm Description

The selected algorithm is SLIM: A Lightweight Block Cipher for Internet of Health Things. According to the project brief, SLIM is a symmetric lightweight block cipher based on the Feistel structure. The same secret key is used for encryption and decryption, while round keys are generated from the original key by a key scheduling process.

- XOR operation: combines data with the round key.
- S-box substitution: replaces small bit groups using a lookup table to provide confusion.
- P-box permutation: rearranges bits using a lookup table to provide diffusion.
- Key scheduling: generates a different round key for each round.
- Feistel structure: splits each block into left and right halves, making decryption possible by applying the round keys in reverse order.

3. Block and Data Handling

The user is not restricted to entering only binary or numeric input. The program accepts letters, numbers, spaces, and symbols as plaintext. The plaintext is encoded using UTF-8 and split into 16-bit blocks. A two-byte length field is added before the message so that padding can be removed correctly after decryption.

Step	Description
Input	User enters a plaintext message and a secret key.
Encoding	Plaintext is converted into UTF-8 bytes.
Blocking	Bytes are grouped into 16-bit blocks.
Padding	If the final block is incomplete, a zero byte is added.
Recovery	After decryption, the stored length is used to remove padding and recover the exact original text.

4. Python Implementation

4.1 S-box Substitution

The S-box follows the lookup-table idea from Lab 3. Each 8-bit half-block is split into four 2-bit chunks. Each chunk is substituted using the table [1, 0, 3, 2], then the chunks are combined again.

4.2 P-box Permutation

The permutation layer uses the Lab 3 P-box table [5, 6, 7, 2, 3, 1, 4, 0]. The input byte is converted to an 8-bit binary string, bits are selected according to the table, the result is reversed as shown in the lab sheet, and the final value is converted back to an integer.

4.3 Key Schedule

The secret key is converted into a 32-bit internal key state. For each of the 16 rounds, the key state is rotated and mixed with a round constant. An 8-bit round key is then derived from the current key state and used by the round function.

4.4 Encryption and Decryption

Each 16-bit block is divided into an 8-bit left half and an 8-bit right half. During encryption, the right half is processed by the round function and XORed with the left half. The halves are then swapped. Decryption uses the same round function but applies the round keys in reverse order.

1. Split the 16-bit plaintext block into left and right 8-bit halves.
2. Generate 16 round keys from the user secret key.
3. For each round, XOR the right half with the round key.
4. Apply the S-box substitution layer.
5. Apply the P-box permutation layer.
6. XOR the round-function output with the left half and swap halves.
7. After all rounds, combine the halves to produce the ciphertext block.
8. For decryption, repeat the Feistel process using the round keys in reverse order.

5. Program Execution and Output

The program displays a menu so the user can encrypt and decrypt multiple messages without the program closing after one run. The screenshot below shows the program accepting mixed plaintext containing letters, numbers, a symbol, and a space.

Program Execution Output

```
SLIM Lightweight Block Cipher - Educational Implementation
1. Encrypt and decrypt a plaintext message
2. Run sample test cases
3. Exit

Choose an option:
Enter any plaintext message: letters, numbers, or symbols.
The program converts the message into 16-bit blocks internally.

Plaintext: Secret key:
Plaintext        : CSE461! 123
Plaintext blocks : 000B 4353 4534 3631 2120 3132 3300
Ciphertext blocks: 14B7 4C81 2BAD 2EDE 3AD8 28DA 1AC2
Recovered        : CSE461! 123
Result           : Decryption recovered the original plaintext.

SLIM Lightweight Block Cipher - Educational Implementation
1. Encrypt and decrypt a plaintext message
2. Run sample test cases
3. Exit

Choose an option:
Sample Test Cases
-----------------------------------------------------------------------------------------------------
Plaintext         Key         Ciphertext Blocks (Hex)                                  Recovered
-----------------------------------------------------------------------------------------------------
HELLO             health      14F0 5498 58D8 11BA                                      HELLO
CSE461            cse461      42EC 179F 70B3 75C0                                      CSE461
```

6. Test Cases

Plaintext	Key	Ciphertext Blocks (Hex)	Recovered Plaintext
HELLO	health	14F0 5498 58D8 11BA	HELLO
CSE461	cse461	42EC 179F 70B3 75C0	CSE461
12345	slim	FD62 CA49 C94D F852	12345
IoHT security!	security	7CB7 18D2 268D 01EA 11BF 05BE 06A1 5384	IoHT security!
CSE461! 123	secret	14B7 4C81 2BAD 2EDE 3AD8 28DA 1AC2	CSE461! 123

All test cases show that the recovered plaintext is identical to the original plaintext. This confirms that encryption and decryption are inverse operations for the implemented Feistel cipher.

7. Conclusion

The completed Python program satisfies the project requirements by implementing a lightweight block cipher using XOR operations, S-box substitution, P-box permutation, key scheduling, encryption, and decryption. The implementation also improves usability by allowing the user to enter normal plaintext instead of requiring manual binary input. The test cases demonstrate that ciphertext can be decrypted successfully to recover the original plaintext.

8. References

- CSE461 Computer Security Project Brief, Spring 2026: SLIM: A Lightweight Block Cipher for Internet of Health Things.
- CSE461 Lab No. 3 handout: P-box implementation, S-box implementation, and split/combine bit operations.
- SLIM: A Lightweight Block Cipher for Internet of Health Things, referenced in the project assignment.